



Obsidian Brain

Sistema RAG locale che risponde a domande basandosi esclusivamente sul contenuto di una vault Obsidian

[Documentazione Tecnica](#)

1. Introduzione e obiettivo del progetto

Obsidian Brain è un sistema di Retrieval-Augmented Generation (RAG) che indicizza il contenuto di una vault Obsidian e risponde a domande basandosi esclusivamente su quel contenuto, rifiutando esplicitamente di rispondere quando l'informazione richiesta non è presente nella vault.

Il progetto è stato sviluppato con l'obiettivo di dimostrare competenze pratiche end-to-end nella costruzione di un sistema di AI engineering completo: parsing di dati semi-strutturati, chunking, generazione di embedding, ricerca vettoriale, integrazione con un modello linguistico locale, esposizione tramite API, e un processo di calibrazione basato su dati reali anziché su parametri scelti arbitrariamente.

1.1 Cosa fa il sistema

- Analizza una cartella di note Obsidian (file `.md`), estraendo frontmatter YAML, wikilink e tag.
- Divide le note in frammenti (chunk) di dimensione gestibile, preservando la struttura delle sezioni Markdown.
- Calcola una rappresentazione numerica (embedding) di ogni chunk, interamente in locale.
- Dato un testo di domanda, recupera i chunk semanticamente più vicini e ne valuta la pertinenza tramite una soglia di distanza.
- Se il contenuto trovato è pertinente, genera una risposta in linguaggio naturale basata solo su quel contenuto, citando la nota di origine.
- Se non trova nulla di sufficientemente pertinente, risponde esplicitamente di non saperlo, invece di rispondere con conoscenza generale del modello linguistico.
- Espone l'intera funzionalità tramite un'API HTTP, con documentazione interattiva automatica.

1.2 Cosa il sistema NON fa (limiti di scope volutamente esclusi)

- Non aggiorna l'indice in modo incrementale: a ogni riavvio, l'intera vault viene ri-processata da zero (vedi sezione 8).
- Non usa alcuna API cloud a pagamento: sia l'embedding sia la generazione della risposta girano in locale, per rendere il progetto liberamente eseguibile da chiunque lo clona, senza chiavi API né costi (vedi sezione 5.1 e 5.2 per la motivazione).
- Non effettua alcun tipo di fine-tuning del modello di embedding o del modello linguistico sul contenuto specifico della vault.

2. Stack tecnologico

Componente	Tecnologia	Ruolo nel progetto
Linguaggio	Python 3.14	Linguaggio principale per l'intera pipeline
IDE	PyCharm	Ambiente di sviluppo
Parsing vault	python-frontmatter, PyYAML	Estrazione frontmatter YAML dalle note Markdown
Embedding testuale	sentence-transformers (paraphrase-multilingual-MiniLM-L12-v2)	Generazione locale degli embedding, multilingua
Vector store	ChromaDB	Persistenza e ricerca per similarità coseno
Generazione risposta	Ollama (llama3.2)	Modello linguistico locale per la risposta finale
API layer	FastAPI, Uvicorn	Esposizione HTTP, validazione richieste, documentazione automatica
Testing	pytest	47 test automatici su ogni componente della pipeline

3. Struttura del progetto

```
obsidian-brain/
├── obsidian_brain/
│   ├── vault_parser.py    parsing frontmatter, wikilink, tag
│   ├── chunker.py        divisione in chunk con header path
│   ├── models.py         dataclass Note e Chunk
│   ├── embedder.py       wrapper su sentence-transformers
│   ├── vector_store.py   persistenza ChromaDB
│   ├── retriever.py      retrieval con soglia di rilevanza
│   ├── responder.py      generazione risposta via Ollama
│   ├── api.py            layer HTTP FastAPI
│   ├── indexer.py        pipeline completa
│   └── inspect_index.py  diagnostica e calibrazione
├── tests/                47 test automatici, uno per modulo
├── sample_vault/         fixture minimale usata dai test
├── demo_vault/           vault di esempio più ricca, per demo
├── requirements.txt
└── README.md
```

4. Pipeline dettagliata

4.1 Parsing della vault

Ogni file `.md` viene letto e separato in frontmatter (metadati YAML) e corpo del testo. Vengono estratti i wikilink nella forma `[[Nota]]` o `[[Nota|alias]]`, e i tag sia dal frontmatter sia dal testo (sintassi `#tag`). Le cartelle interne di Obsidian (`.obsidian`, `.trash`) vengono escluse automaticamente.

4.2 Chunking

Il contenuto di ogni nota viene prima diviso per sezione Markdown (`##`, `###` ...), preservando un "percorso" di header (es. `Setup > Installazione`) come metadato di ogni chunk. Solo se una sezione supera la dimensione massima configurata (800 caratteri di default) viene ulteriormente divisa per paragrafo, con una sovrapposizione (overlap) di 100 caratteri tra chunk consecutivi per non perdere contesto a cavallo del taglio.

4.3 Embedding e vector store

Ogni chunk viene trasformato in un vettore numerico tramite `sentence-transformers`, e salvato in una collection ChromaDB persistente su disco, configurata per usare la distanza coseno (invece del default euclideo di ChromaDB) — un valore compreso sempre tra 0

(identico) e 2 (opposto), indipendente dalla scala degli embedding, che rende interpretabile la soglia di rilevanza dello step successivo.

4.4 Retrieval con soglia di rilevanza

Data una domanda, il sistema recupera i chunk più vicini semanticamente e scarta quelli la cui distanza supera una soglia configurabile (`DEFAULT_MAX_DISTANCE = 0.6` , calibrata empiricamente — vedi sezione 7). Se nessun chunk supera la soglia, il retrieval segnala esplicitamente "non trovato", senza restituire risultati deboli o fuorvianti.

4.5 Generazione della risposta

Solo se il retrieval ha trovato contenuto pertinente, i chunk vengono formattati in un blocco di contesto (con citazione della nota di origine) e passati a un modello linguistico locale (Ollama) insieme a un prompt di sistema che impone di rispondere esclusivamente sulla base del contesto fornito. Se il retrieval non ha trovato nulla, il modello linguistico non viene nemmeno interpellato: zero chiamate inutili, zero rischio di allucinazione quando la vault non ha la risposta.

4.6 API HTTP

L'intera pipeline è esposta tramite due endpoint FastAPI: `GET /health` (stato del servizio e numero di chunk indicizzati) e `POST /ask` (la domanda, con validazione automatica che rifiuta domande vuote). FastAPI genera automaticamente una documentazione interattiva (Swagger UI) su `/docs` .

5. Scelte tecniche e motivazioni

5.1 Embedding locale invece di un'API a pagamento

È stato scelto `sentence-transformers` al posto di un'API di embedding a pagamento (es. Voyage AI). La motivazione è specifica per un progetto portfolio: chiunque clona il repository può eseguirlo immediatamente, senza doversi procurare una chiave API né sostenere alcun costo. Il modello scelto, `paraphrase-multilingual-MiniLM-L12-v2` , è multilingua e gestisce correttamente sia l'italiano sia l'inglese.

5.2 Ollama invece di un'API LLM cloud

Per lo stesso motivo, la generazione della risposta finale usa Ollama (modello `llama3.2`) invece di un'API cloud come quella di Anthropic, valutata inizialmente. Il compromesso accettato è che l'utente deve installare Ollama e scaricare una volta i pesi del modello (circa 2 GB): un costo una tantum, a fronte di un utilizzo completamente gratuito e offline in seguito.

5.3 Distanza coseno invece del default euclideo

ChromaDB usa di default la distanza euclidea (L2), la cui scala dipende dalla norma dei vettori di embedding e non ha un intervallo fisso: nei primi test questo produceva valori come 15-29, di fatto impossibili da interpretare per fissare una soglia di rilevanza sensata. La collection è stata configurata esplicitamente per usare la distanza coseno, sempre compresa tra 0 e 2, rendendo la soglia interpretabile e trasferibile tra vault diverse.

5.4 Chunking gerarchico

Il contenuto viene diviso prima per struttura logica (sezioni Markdown) e solo secondariamente per dimensione, invece di tagliare a blocchi di lunghezza fissa senza riguardo per il contenuto. Questo evita di spezzare un discorso coerente a metà, e permette di allegare a ogni chunk un "percorso" di header utile sia per il debug sia per una futura citazione più precisa della fonte.

5.5 Dependency injection sistematica

Ogni componente che dipende da una risorsa esterna (il modello di embedding, il client di chat con Ollama, il vector store) la riceve come parametro iniettabile invece di crearla internamente. Questo pattern, applicato in modo sistematico, permette di testare tutta la logica di business con dipendenze finte (`FakeEmbedder`, `FakeChatClient`, `embedder` "scriptati" con vettori scelti a mano), senza mai scaricare modelli reali o effettuare chiamate di rete durante l'esecuzione dei test.

5.6 Separazione tra costruzione dell'app e costruzione dell'indice

Nel layer API, la funzione che costruisce l'app FastAPI (`create_app`) è separata da quella che indicizza la vault e assembla le dipendenze reali (`build_app`). La costruzione "pesante" avviene solo quando il modulo viene eseguito direttamente, non quando viene importato: questo evita che l'importazione del modulo nei test scateni un'indicizzazione completa come effetto collaterale, mantenendo la suite di test veloce e isolata.

6. Bug reali risolti durante lo sviluppo

Il processo di sviluppo ha incontrato diversi problemi concreti, tipici di un progetto Python fatto girare su Windows con una versione di Python molto recente. Vengono documentati perché rappresentano un processo di debugging sistematico, non solo il risultato finale.

6.1 Compatibilità di Python 3.14 con pacchetti compilati

Alcune dipendenze (in particolare `tokenizers`, usata da `sentence-transformers`) non disponevano ancora di un pacchetto precompilato per Python 3.14, uscito da pochissimo tempo al momento dello sviluppo. Il tentativo di installazione provava a compilare la

libreria da sorgente tramite Rust, fallendo per incompatibilità tra il compilatore e la versione di Python. La soluzione è stata rimuovere il vincolo di versione esatta (`==`) sui pacchetti coinvolti, sostituendolo con un vincolo minimo (`>=`), permettendo a pip di scegliere una versione più recente già dotata di un pacchetto precompilato compatibile.

6.2 Codifica dei file e fine riga su Windows

Alcuni file di test creati tramite copia-incolla risultavano salvati su un'unica riga, senza gli a-capo attesi, impedendo il corretto riconoscimento del blocco frontmatter YAML. La diagnosi è stata effettuata ispezionando i byte grezzi del file (`open(file, "rb").read()`), confermando l'assenza dei caratteri di a-capo. La lettura dei file è stata inoltre resa più robusta usando la codifica `utf-8-sig` invece di `utf-8`, per gestire automaticamente un eventuale BOM (Byte Order Mark) presente nei file salvati da alcuni editor su Windows.

6.3 Separatore di percorso Windows vs Unix

Il percorso relativo di ogni nota, usato come identificatore stabile, veniva costruito con il separatore nativo del sistema operativo (`\` su Windows, `/` su Unix), causando fallimenti nei test che si aspettavano un formato coerente. Risolto forzando l'uso di `.as_posix()`, che restituisce sempre il formato con forward slash indipendentemente dal sistema operativo.

6.4 Conflitto case-insensitive nel .gitignore

Una regola del `.gitignore` pensata per escludere la cartella della vault personale dell'utente (`Obsidian_Brain/`) ha accidentalmente escluso anche il pacchetto Python del progetto (`obsidian_brain/`), poiché Git su Windows tratta i nomi di file e cartelle senza distinguere maiuscole/minuscole. Risolto rinominando la cartella della vault personale con un nome privo di ambiguità.

6.5 Metrica di distanza non riapplicata dopo il reset dell'indice

Il metodo `VectorStore.reset()`, usato per ricostruire l'indice da zero, cancellava la collection ChromaDB esistente e ne ricreava una nuova con lo stesso nome, ma senza ripassare la configurazione della metrica di distanza coseno. Il risultato era che, dopo un reset, ChromaDB tornava silenziosamente alla metrica euclidea di default, vanificando la calibrazione della soglia di rilevanza. Il bug è stato individuato confrontando le distanze attese (comprese tra 0 e 2) con quelle osservate (valori a due cifre, tipici della distanza euclidea) durante un test end-to-end reale. Risolto salvando la configurazione della metrica come attributo dell'istanza e riapplicandola esplicitamente anche nel metodo `reset()`.

7. Calibrazione empirica (real-world testing)

I parametri di chunking e di soglia di rilevanza sono stati inizialmente scelti come stime ragionevoli, non come valori arbitrari definitivi. Sono stati poi validati eseguendo il sistema su una vault con note realistiche (paragrafi multi-frase strutturati in sezioni, non singole righe), contro un campione di 11 domande reali, di cui 9 con risposta attesa nella vault e 2 deliberatamente fuori tema.

Tipo di domanda	Distanza coseno migliore (top-1)
Domande pertinenti (9 su 11)	tra 0.252 e 0.582
Domande fuori tema (2 su 11)	tra 0.763 e 1.033

Il margine di sicurezza tra i due gruppi (circa 0.18, nell'intervallo 0.582–0.763) non contiene nessuna domanda del campione: qualunque soglia in quell'intervallo, inclusa quella scelta (0.6), separa perfettamente i casi pertinenti da quelli fuori tema, senza falsi positivi né falsi negativi sul campione testato.

Un test preliminare con note costituite da singole frasi molto brevi (poche parole ciascuna) aveva invece mostrato una sovrapposizione tra i due gruppi, con alcune domande pertinenti a distanza maggiore di alcune domande fuori tema — a conferma che la qualità del retrieval dipende in modo significativo dalla ricchezza di contesto delle note originali, non solo dai parametri numerici del sistema.

8. Limiti noti e sviluppi futuri

- **Note molto brevi:** con note fatte di singole frasi isolate, il margine tra domande pertinenti e non pertinenti si riduce, aumentando il rischio di falsi positivi o rifiuti indebiti (vedi sezione 7).
- **Nessun aggiornamento incrementale dell'indice:** a ogni avvio, l'intera vault viene ri-processata e re-indicizzata da zero. Accettabile per vault di dimensioni moderate (decine o centinaia di note), da rivedere per vault molto grandi, dove converrebbe ricalcolare solo i chunk delle note modificate dall'ultima indicizzazione.
- **Un solo modello di embedding generico:** per un dominio molto specialistico (es. terminologia tecnica di nicchia), un modello di embedding fine-tuned sul dominio specifico potrebbe ottenere risultati di retrieval migliori.
- **Citazioni a livello di nota, non di paragrafo esatto:** il sistema cita correttamente la nota di origine (e, quando disponibile, la sezione), ma non evidenzia la frase esatta all'interno del chunk usata per generare la risposta.
- **Deploy:** il progetto è pensato per l'esecuzione in locale. Un possibile sviluppo futuro è la containerizzazione (Docker) per semplificare ulteriormente l'esecuzione su macchine diverse.

9. Conclusioni

Obsidian Brain copre l'intero ciclo di sviluppo di un sistema RAG applicato: parsing di dati semi-strutturati, chunking consapevole della struttura del documento, generazione e persistenza di embedding vettoriali, retrieval con soglia di rilevanza calibrata empiricamente, integrazione con un modello linguistico locale secondo un principio di grounding rigoroso, ed esposizione tramite un'API HTTP documentata. Le scelte tecniche adottate sono state motivate e documentate, così come i problemi reali incontrati durante lo sviluppo e la loro risoluzione, con l'obiettivo di presentare un progetto completo, funzionante e tecnicamente consapevole dei propri limiti.

Documento generato come parte della documentazione tecnica del progetto Obsidian Brain.